Types of Learning

Instructors: Dr. Durgesh Singh

CSE Discipline, PDPM IIITDM, Jabalpur -482005

Linear Regression

- Linear Regression is a supervised machine learning algorithm.
- It tries to find out the best linear relationship that describes the data you have i.e. .

  Given: (X, Y) training examples

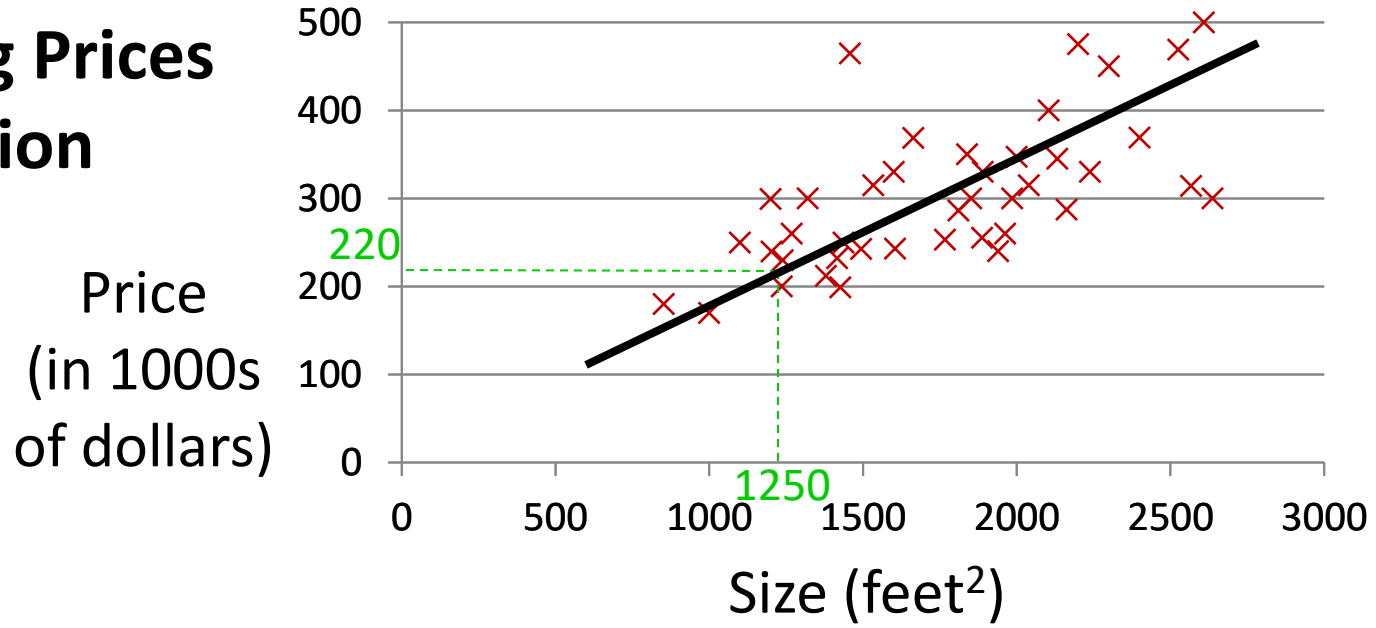
  Goal: find a (hypothesis) function (h) which predicts y for given x
- In regression y is continuous value i.e., real numbers.
- Input feature X can be comprise of a single feature or multiple feature

Linear Regression

- It works in a such that it finds the best fit line from the given data.
- So, the goal of linear regression is to fit a line through a set of points.
  - Linear regression looks at entire data points and plots a best-fit line.

Linear regression with one variable

Ex. Housing Prices prediction



Supervised Learning

Given the "right answer" for each example in the data.

Regression Problem

Predict real-valued output

Classification : Discrete-valued output

Training set of housing prices

Size in feet² (x)	Price ($) in 1000's (y)
2104	460
1416	232
1534	315
852	178
…	…

m

Notation:

m = Number of training examples

x's = "input" variable / features

y's = "output" variable / "target" variable

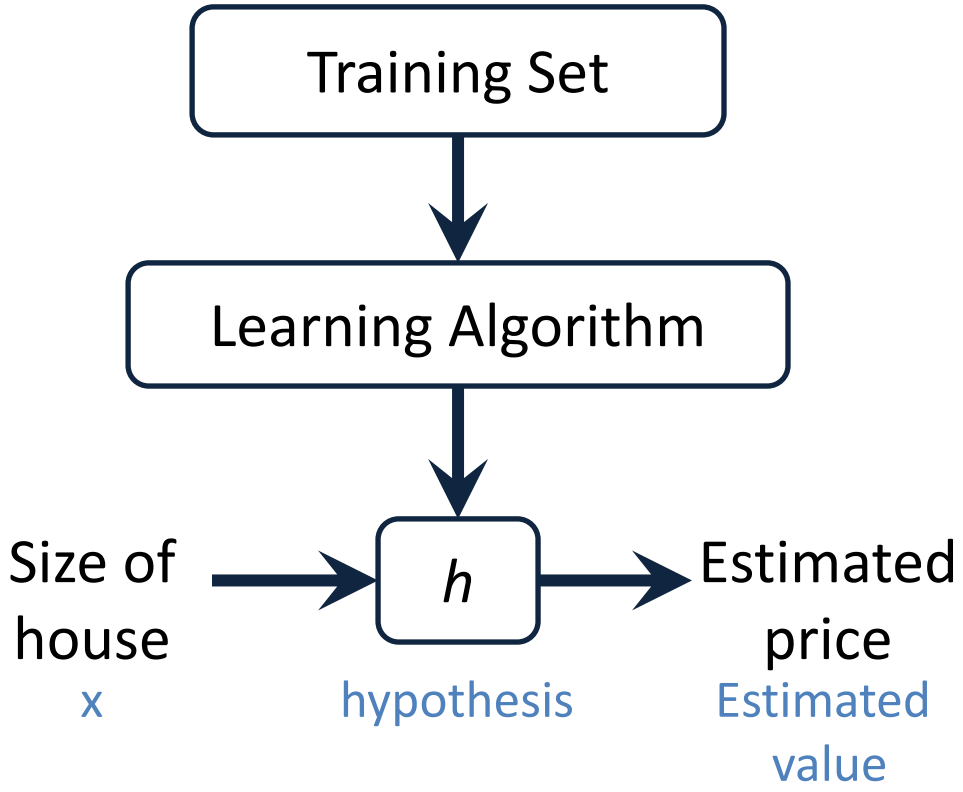
$x^{(1)} = 2104$

$x^{(2)} = 1416$

$y^{(1)} = 460$

(x, y) – one training example

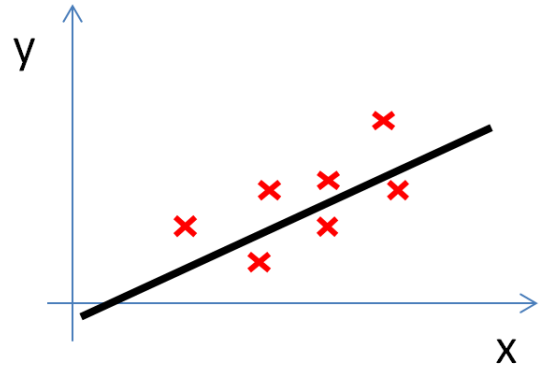
$(x^{(i)}, y^{(i)})$ – ith training example



h maps from x's to y's

How do we represent *h* ?

$h_\Theta(x) = \Theta_0 + \Theta_1 x$



Linear regression with one variable.

Univariate linear regression.

One variable

Linear regression with one variable

Cost function

Training Set

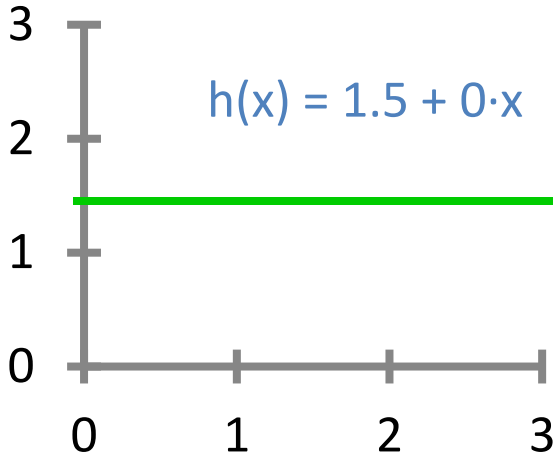
Size in feet² (x)	Price ($) in 1000's (y)
2104	460
1416	232
1534	315
852	178
…	…

Hypothesis: $h_\theta(x) = \theta_0 + \theta_1 x$

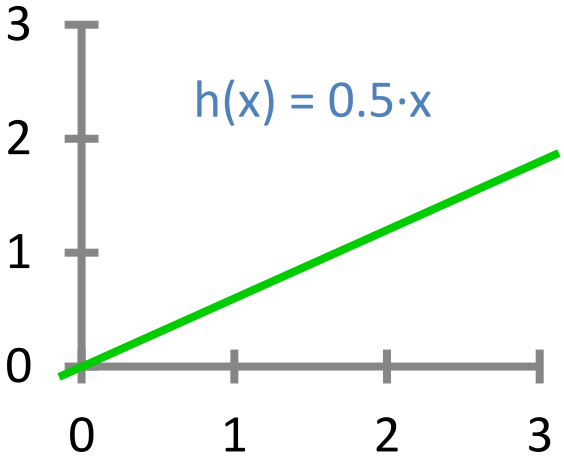
θ_i's: Parameters

How to choose θ_i's ?

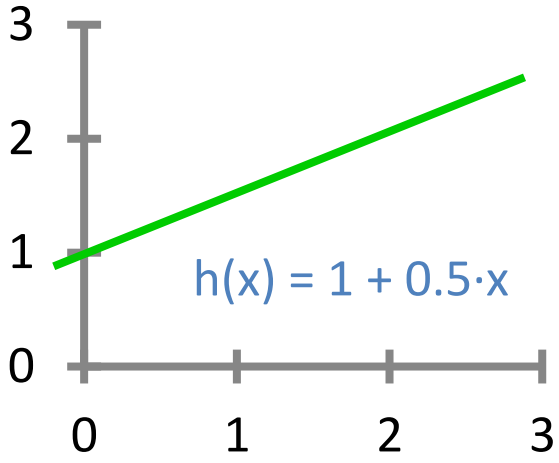
$$h_\theta(x) = \theta_0 + \theta_1 x$$



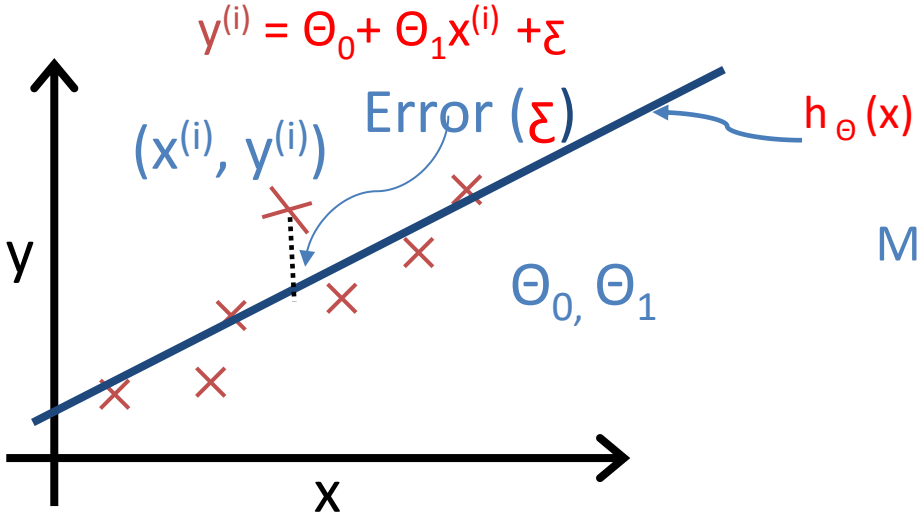
$\theta_0 = 1.5$
$\theta_1 = 0$



$\theta_0 = 0$
$\theta_1 = 0.5$



$\theta_0 = 1$
$\theta_1 = 0.5$



$$\underset{\Theta_0\,\Theta_1}{\text{Minimize}} \; \frac{1}{2m}\sum_{i=1}^{m} (h_\Theta(x^{(i)}) - y^{(i)})^2$$

$$h_\Theta(x) = \Theta_0 + \Theta_1 x^{(i)}$$

$$J(\Theta_0, \Theta_1) = \frac{1}{2m}\sum_{i=1}^{m} (h_\Theta(x^{(i)}) - y^{(i)})^2$$

Idea: Choose θ_0, θ_1 so that $h_\theta(x)$ is close to y for our training examples (x, y)

$$\underset{\Theta_0\,\Theta_1}{\text{Minimize}}\; J(\Theta_0, \Theta_1) : \text{Cost Function}$$

Squared error function

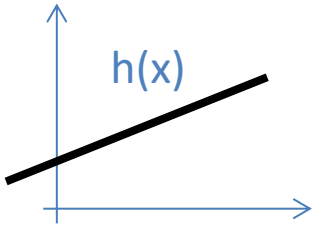
Simplified

Hypothesis:

$$h_\theta(x) = \theta_0 + \theta_1 x$$

Parameters:

$$\theta_0, \theta_1$$



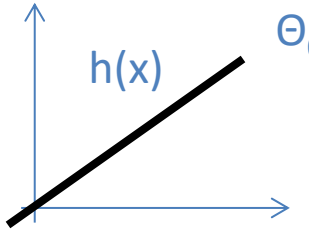
Cost Function:

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^{m} \left(h_\theta(x^{(i)}) - y^{(i)} \right)^2$$

Goal: $\underset{\theta_0, \theta_1}{\text{minimize}}\ J(\theta_0, \theta_1)$

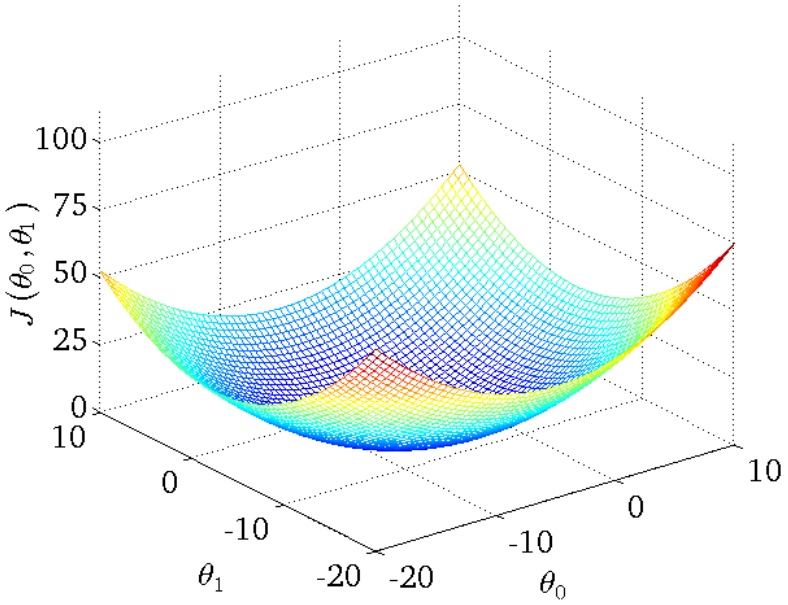
$$h_\theta(x) = \theta_1 x$$

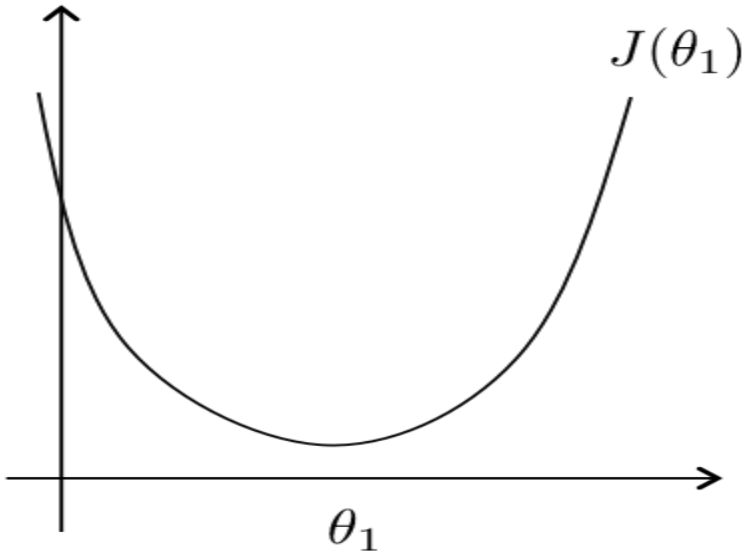
$$\theta_1$$



$$J(\theta_1) = \frac{1}{2m} \sum_{i=1}^{m} \left(h_\theta(x^{(i)}) - y^{(i)} \right)^2$$

$\underset{\theta_1}{\text{minimize}}\ J(\theta_1)$


100
75
50
25
0
$J(\theta_0, \theta_1)$
10
0
-10
-20
θ_1
-20
-10
0
10
θ_0


$J(\theta_1)$
θ_1

Hypothesis: $h_\theta(x) = \theta_0 + \theta_1 x$

Parameters: θ_0, θ_1

Cost Function: $J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^{m} \left(h_\theta(x^{(i)}) - y^{(i)} \right)^2$

Goal: $\underset{\theta_0, \theta_1}{\text{minimize}}\ J(\theta_0, \theta_1)$

Linear regression with one variable

Gradient descent

Have some function $J(\theta_0, \theta_1)$

Want $\min_{\theta_0, \theta_1} J(\theta_0, \theta_1)$

Outline:

- Start with some θ_0, θ_1
- Keep changing θ_0, θ_1 to reduce $J(\theta_0, \theta_1)$

  until we hopefully end up at a minimum

Gradient descent algorithm

repeat until convergence {

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1) \qquad (\text{for } j = 0 \text{ and } j = 1)$$

}

Learning rate

Simultaneously update
Θ_0 & Θ_1

Correct: Simultaneous update

$$\text{temp0} := \theta_0 - \alpha \frac{\partial}{\partial \theta_0} J(\theta_0, \theta_1)$$
$$\text{temp1} := \theta_1 - \alpha \frac{\partial}{\partial \theta_1} J(\theta_0, \theta_1)$$
$$\theta_0 := \text{temp0}$$
$$\theta_1 := \text{temp1}$$

Incorrect:

$$\text{temp0} := \theta_0 - \alpha \frac{\partial}{\partial \theta_0} J(\theta_0, \theta_1)$$
$$\theta_0 := \text{temp0}$$
$$\text{temp1} := \theta_1 - \alpha \frac{\partial}{\partial \theta_1} J(\theta_0, \theta_1)$$
$$\theta_1 := \text{temp1}$$

Gradient descent algorithm

repeat until convergence {

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1)$$

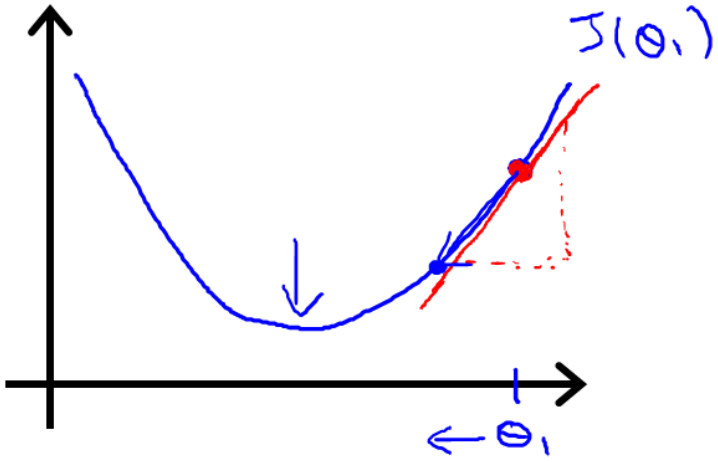
(simultaneously update $j = 0$ and $j = 1$)

}

Learning rate

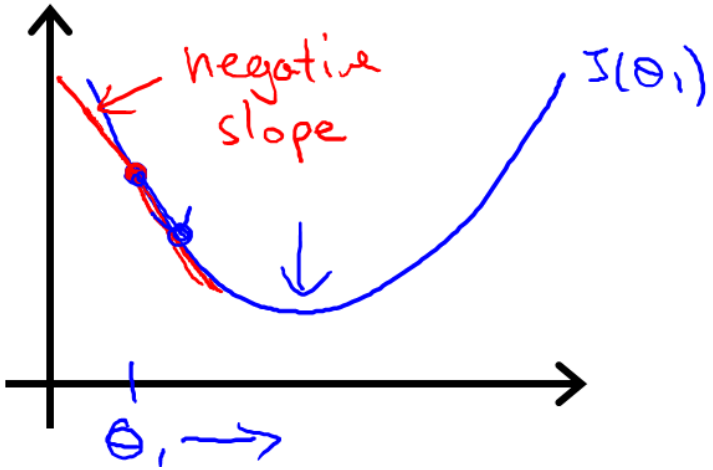
derivative

The partial derivative is the slop of the tangent line to this curve at that point.



$$\Theta_1 := \Theta_1 - \alpha \underbrace{\frac{\partial}{\partial \theta_1} J(\theta_1)}_{\geq 0}$$

$$\Theta_1 := \Theta_1 - \alpha(positive\ number)$$

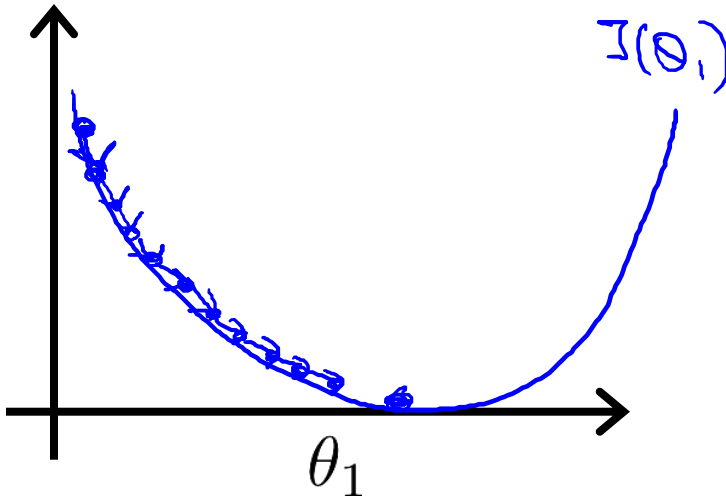


$$\Theta_1 := \Theta_1 - \alpha \underbrace{\frac{\partial}{\partial \theta_1} J(\theta_1)}_{\leq 0}$$

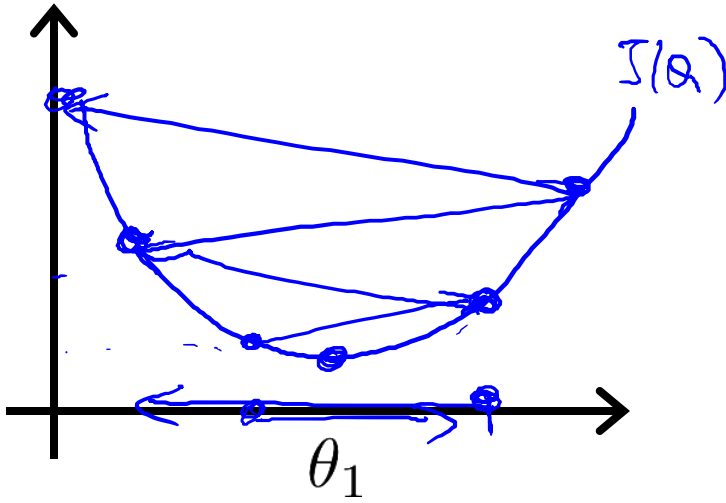
$$\Theta_1 := \Theta_1 - \alpha(negative\ number)$$

$$\theta_1 := \theta_1 - \alpha \frac{\partial}{\partial \theta_1} J(\theta_1)$$

If α is too small, gradient descent can be slow.



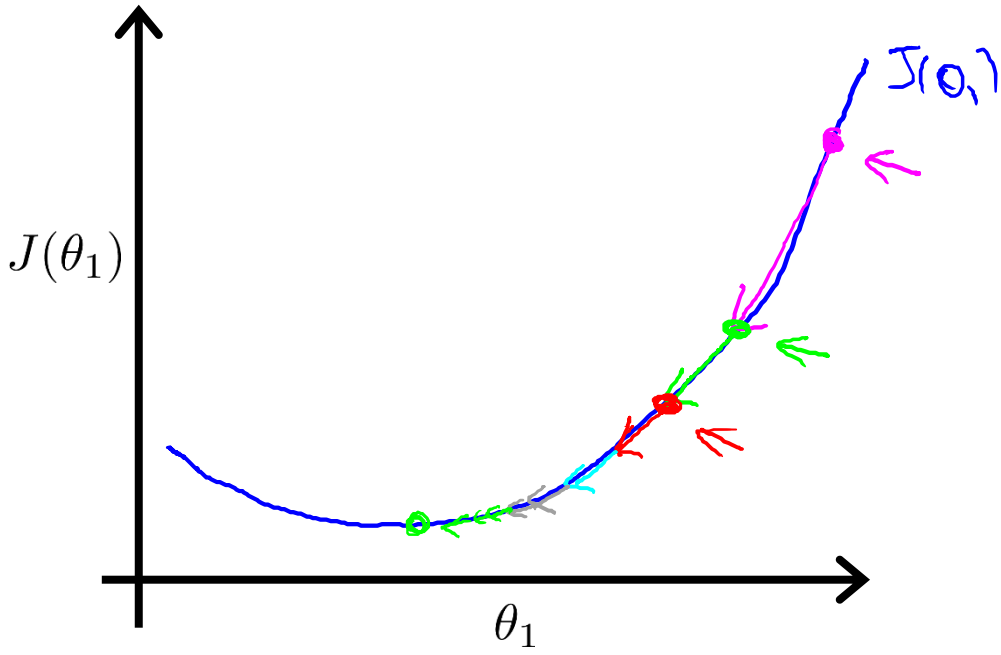
If α is too large, gradient descent can overshoot the minimum. It may fail to converge, or even diverge.



Gradient descent can converge to a local minimum, even with the learning rate α fixed.

$$\theta_1 := \theta_1 - \alpha \frac{\partial}{\partial \theta_1} J(\theta_1)$$

As we approach a local minimum, gradient descent will automatically take smaller steps. So, no need to decrease α over time.



Linear regression with one variable

Gradient descent for linear regression

Gradient descent algorithm

repeat until convergence {

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1)$$

(for $j = 1$ and $j = 0$)

}

Linear Regression Model

$$h_\theta(x) = \theta_0 + \theta_1 x$$

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^{m} \left(h_\theta(x^{(i)}) - y^{(i)} \right)^2$$

$$\frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1) = \frac{\partial}{\partial \theta_j} \cdot \frac{1}{2m} \sum_{i=1}^{m} (h_\Theta(x^{(i)}) - y^{(i)})^2$$

$$= \frac{\partial}{\partial \theta_j} \cdot \frac{1}{2m} \sum_{i=1}^{m} (\theta_0 + \theta_1 \cdot x^{(i)} - y^{(i)})^2$$

$$j = 0 : \frac{\partial}{\partial \theta_0} J(\theta_0, \theta_1) = \frac{1}{m} \sum_{i=1}^{m} (h_\Theta(x^{(i)}) - y^{(i)})$$

$$j = 1 : \frac{\partial}{\partial \theta_1} J(\theta_0, \theta_1) = \frac{1}{m} \sum_{i=1}^{m} (h_\Theta(x^{(i)}) - y^{(i)}) \cdot x^{(i)}$$

Gradient descent algorithm

repeat until convergence {

$$\theta_0 := \theta_0 - \alpha \frac{1}{m} \sum_{i=1}^{m} \left(h_\theta(x^{(i)}) - y^{(i)} \right)$$

$$\theta_1 := \theta_1 - \alpha \frac{1}{m} \sum_{i=1}^{m} \left(h_\theta(x^{(i)}) - y^{(i)} \right) \cdot x^{(i)}$$

}

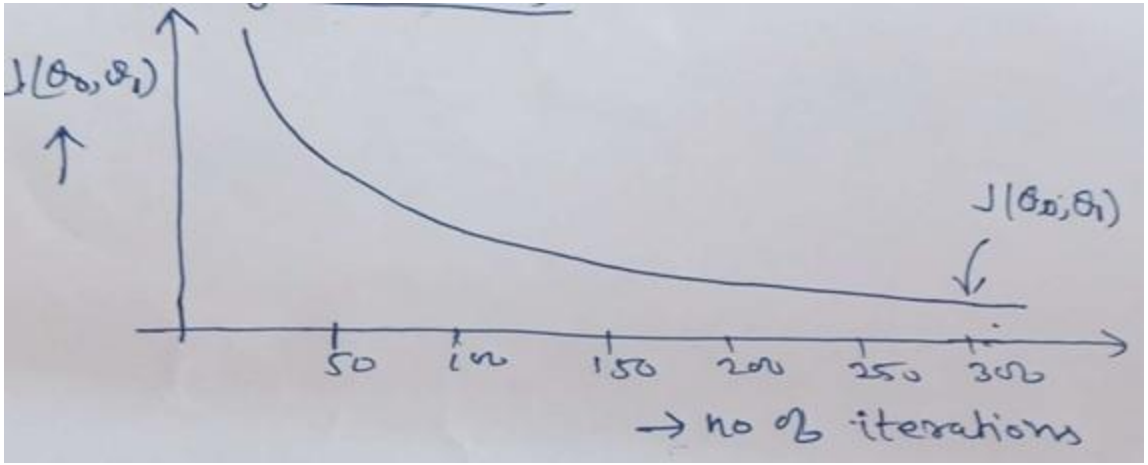
$\frac{d}{d\theta_0} \cdot J(\theta_0, \theta_1)$

$\frac{d}{d\theta_1} \cdot J(\theta_0, \theta_1)$

update θ_0 and θ_1 simultaneously

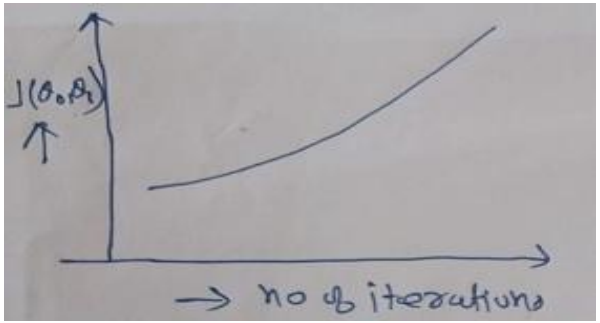
How to choose learning rate (α)

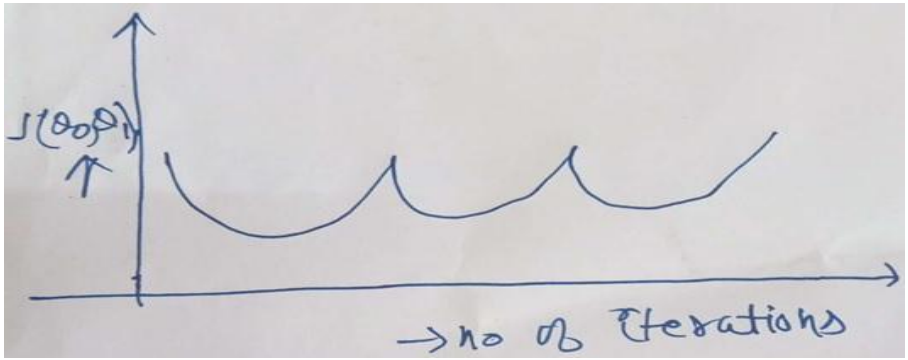
If gradient decent is working correctly then $J(\theta_0, \theta_1)$ should decrease after every iteration.



How to choose learning rate (α)

If gradient decent is not working correctly then..





- **Cause:** Large α
- **Solution:** use small α
- **To choose α, Try as,**

0.001, 0.003, 0.01, 0.03, 0.1, 1.0

Linear Regression with Multiple Variable

- Ex. House Price Prediction

Size in feet^2	No. of bed rooms	No. of floors	Age of House in Years	Price ($) in 1000's (y)
2104	5	1	45	450
1416	3	2	40	232
1534	3	2	30	315
….	…	-----		…
…..	….	----		….

Linear Regression with Multiple Variable

Ex. House Price Prediction

Notation:

n = number of features

$x^{(i)}$ = input (features) of ith training example

$x_j^{(i)}$ =value of feature j in i^{th} training example.

Hypothesis

Simple Linear Regression: $h_\theta(x) = \theta_0 + \theta_1 x$

For multi-variate linear regression:

$$h_\theta(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$$

- For convenience of notation, define $x_0 = 1$
- Hypothesis:

$$h_\theta(x) = \theta^T x = \theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$$

Parameters: $\theta_0, \theta_1, \ldots, \theta_n$

Cost function:

$$J(\theta_0, \theta_1, \ldots, \theta_n) = \frac{1}{2m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)})^2$$

Gradient descent

Repeat $\{$

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \ldots, \theta_n)$$

$\}$ (simultaneously update for every $j = 0, \ldots, n$)

Gradient Descent

Previously (n=1):

Repeat {

$$\theta_0 := \theta_0 - \alpha \frac{1}{m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)})$$

$$\theta_1 := \theta_1 - \alpha \frac{1}{m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)}) x^{(i)}$$

(simultaneously update θ_0, θ_1)

}

New algorithm $(n \geq 1)$:

Repeat {

$$\theta_j := \theta_j - \alpha \frac{1}{m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

simultaneously update θ_j for $j = 0, \ldots, n$

}

Gradient Descent

Previously (n=1):

Repeat $\{$

$$\theta_0 := \theta_0 - \alpha \frac{1}{m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)})$$

$$\theta_1 := \theta_1 - \alpha \frac{1}{m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)}) x^{(i)}$$

(simultaneously update θ_0, θ_1)

$\}$

New algorithm $(n \geq 1)$:

Repeat $\{$

$$\theta_j := \theta_j - \alpha \frac{1}{m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

simultaneously update θ_j for $j = 0, \ldots, n$

$\}$

$$\theta_0 := \theta_0 - \alpha \frac{1}{m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)}) x_0^{(i)}$$

$$\theta_1 := \theta_1 - \alpha \frac{1}{m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)}) x_1^{(i)}$$

$$\theta_2 := \theta_2 - \alpha \frac{1}{m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)}) x_2^{(i)}$$

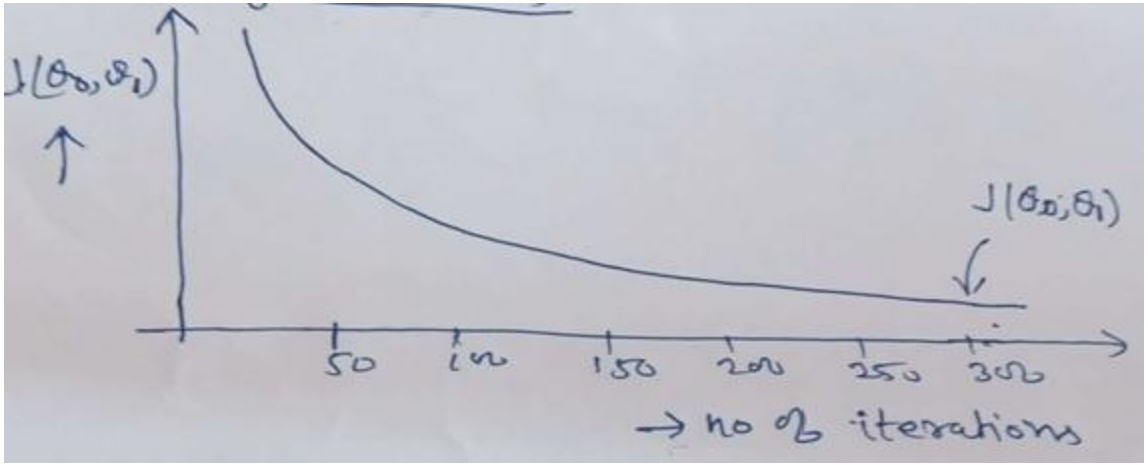
$\cdots$

Is gradient descent working properly?

- Plot how $J(\theta)$ changes with every iteration of gradient descent
- For sufficiently small learning rate, $J(\theta)$ should decrease with every iteration
  - If not, learning rate needs to be reduced
- However, too small learning rate means slow convergence

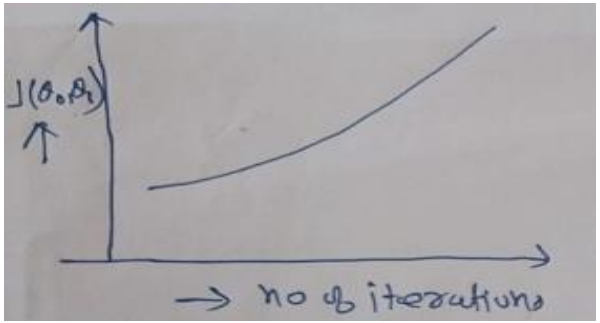
How to choose learning rate (α)

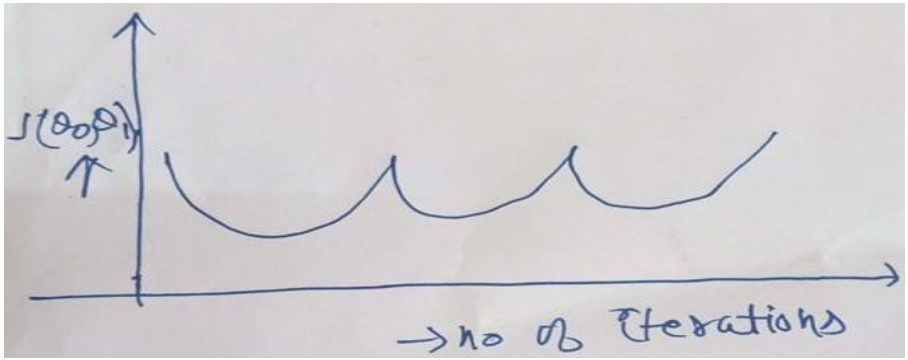
If gradient decent is working correctly then J(θ) should decrease after every iteration.



How to choose learning rate (α)

If gradient decent is not working correctly then..





- **Cause:** Large α
- **Solution:** use small α
- **To choose α, Try as,**

0.001, 0.003, 0.01, 0.03, 0.1, 1.0

When to end gradient descent?

- Declare convergence if $J(\theta)$ decreases by less than 0.001 in an iteration (assuming $J(\theta)$ is decreasing in every iteration)

Logistic Regression

- Logistic regression is a classifier.
- Classification is supervised learning problem.
  - ✓Tumor: malignant or benign cancer
  - ✓Email: Spam or Not spam
- In logistic regression the prediction values always between 0 and 1.

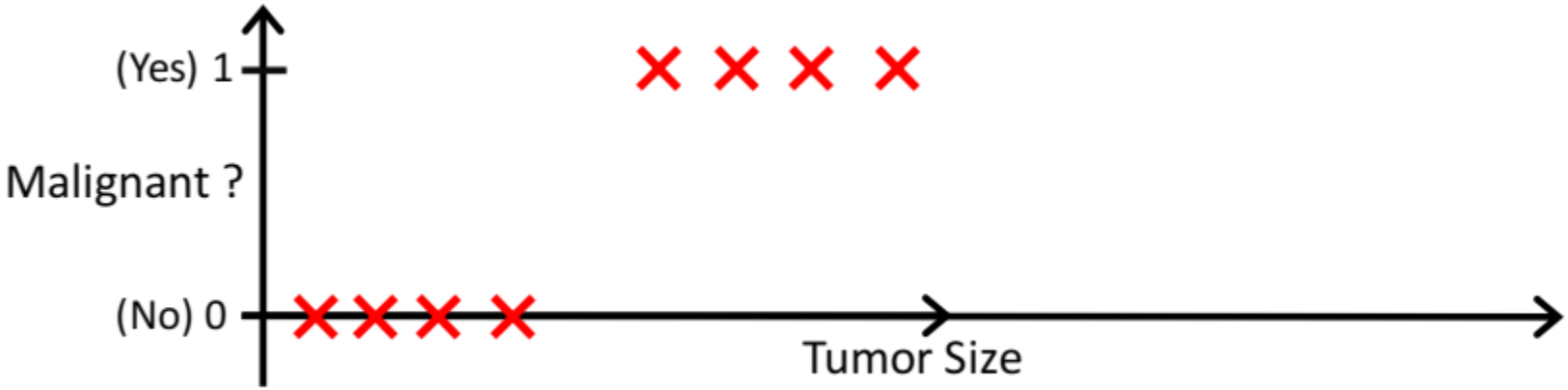
$$0 \leq h_\theta(x) \leq 1$$

Classification Problem

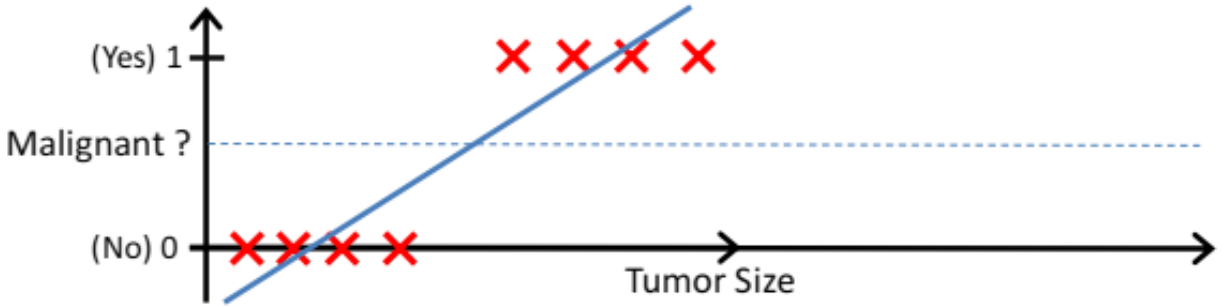
- Classification problem just like the regression problem, except that the values y. We now want to predict take only a small number of discrete values.
- For simplicity, we will focus here on the binary classification problem in which y can take only two values, 0 and 1.
- For instance, if we are trying to build a classifier for tumor, then y may be 1 if it is a malignant, and 0 otherwise.
- 0 is also called the **negative class**, and 1 the **positive class.**
- **Can we solve this using linear regression ?**

Classification Problem

- We could approach the classification problem ignoring the fact that y is discrete-valued, and use our old linear regression algorithm to try to predict y given x.
- However, it is easy to construct examples where this method performs very poorly.



Can we solve the problem using linear regression?

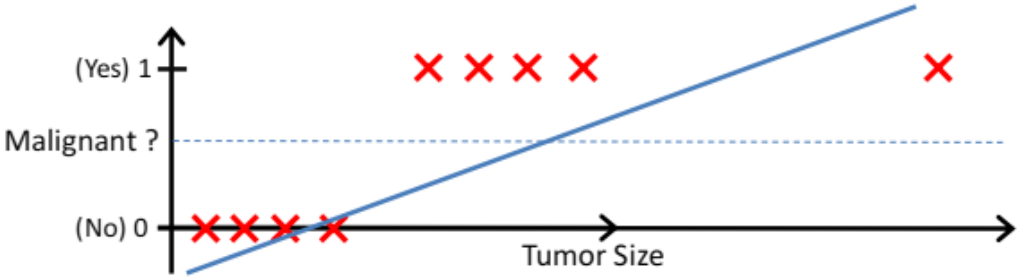


Can we solve the problem using linear regression? E.g., fit a straight line and define a threshold at 0.5

Threshold classifier output $h_\theta(x)$ at 0.5:

If $h_\theta(x) \geq 0.5$, predict "y = 1"

If $h_\theta(x) < 0.5$, predict "y = 0"



Can we solve the problem using linear regression? E.g., fit a straight line and define a threshold at 0.5

Threshold classifier output $h_\theta(x)$ at 0.5:

If $h_\theta(x) \geq 0.5$, predict "y = 1"

If $h_\theta(x) < 0.5$, predict "y = 0"

Failure due to adding a new point

Classification: y = 0 or 1

$h_\theta(x)$ can be > 1 or < 0

Another drawback of using linear regression for this problem

What we need:

Logistic Regression: $0 \leq h_\theta(x) \leq 1$

Logistic Regression

- To fix this, let's change the form for our linear regression hypotheses $h_\Theta(x)$ as

$$h_\theta(x) = g(\theta^T x)$$

$$= \frac{1}{1 + e^{-\theta^T x}}$$

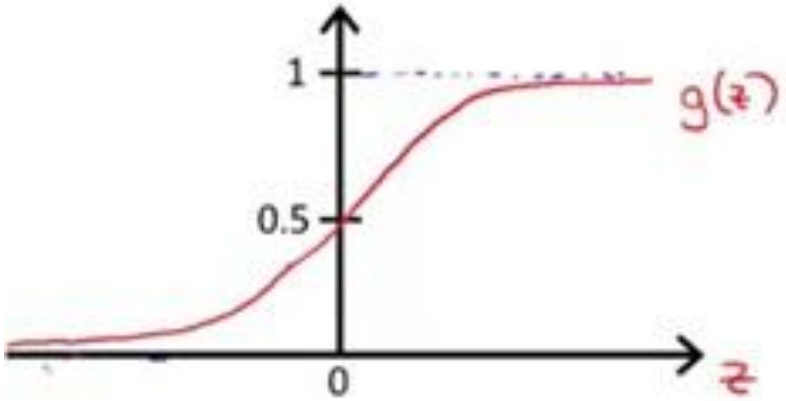
where

$$g(z) = \frac{1}{1 + e^{-z}}$$

is called the logistic function or the sigmoid function.

Sigmoid Function

$$g(z) = \frac{1}{1 + e^{-z}}$$



Interpretation of Hypothesis Output

- In logistic regression, we do not just predict +ve or –ve.
- We predict probability, how likely this is to be +ve or –ve.

$$h_\Theta(x) = P(y=1 \mid x; \theta)$$

- $h_\Theta(x)$ = estimated probability that y = 1 on input x Example: If classifier for tumor gives $h_\Theta(x)$ =0.7 means 70% chance of tumor being malignant.

Cost function for Logistic Regression

- Training Set: $\{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \cdots, (x^{(m)}, y^{(m)})\}$

m examples $\quad x \in \begin{bmatrix} x_0 \\ x_1 \\ \cdots \\ x_n \end{bmatrix} \quad x_0 = 1, y \in \{0, 1\}$

$$h_\theta(x) = \frac{1}{1 + e^{-\theta^T x}}$$

How to choose parameters θ ?

Decision Boundary

- Suppose predict “y=1” if $h_\Theta(x) >= 0.5$

  predict “y=0” if $h_\Theta(x) < 0.5$

Cost function

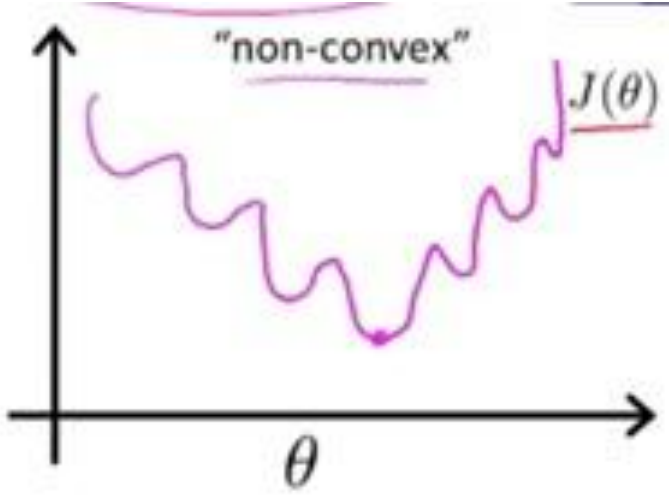
Linear Regression: $J(\theta) = \frac{1}{m} \sum_{i=1}^{m} \frac{1}{2} \left(h_\theta(x^{(i)}) - y^{(i)} \right)^2$

Squared error cost function:

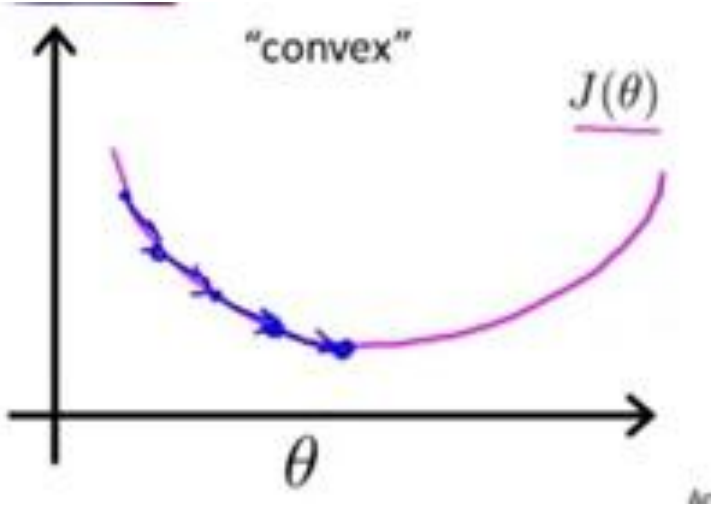
$$\text{Cost}(h_\theta(x^{(i)}), y^{(i)}) = \frac{1}{2} \left(h_\theta(x^{(i)}) - y^{(i)} \right)^2$$

However, this cost function is non-convex for the hypothesis of logistic regression.

Cost function



Logistic regression

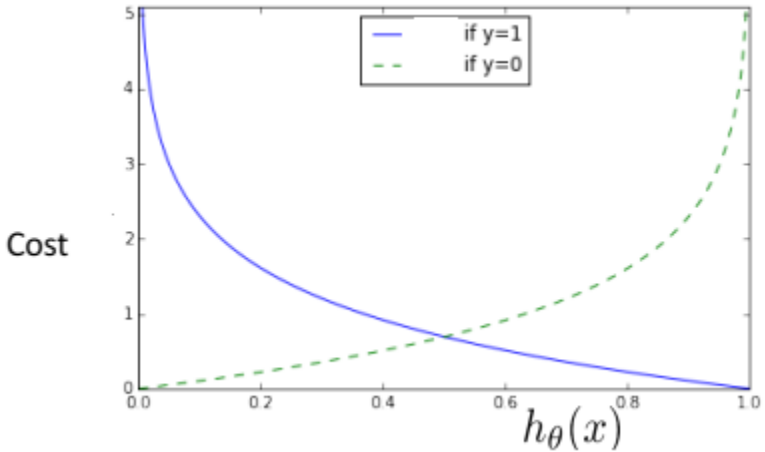


Linear regression

Logistic regression cost function

$$J(\theta) = \frac{1}{m} \sum_{i=1}^{m} \text{Cost}(h_\theta(x^{(i)}), y^{(i)})$$

$$\text{Cost}(h_\theta(x), y) = \begin{cases} -\log(h_\theta(x)) & \text{if } y = 1 \\ -\log(1 - h_\theta(x)) & \text{if } y = 0 \end{cases}$$



Logistic regression cost function

This cost function is convex

$$J(\theta) = \frac{1}{m}\sum_{i=1}^{m} \mathrm{Cost}(h_\theta(x^{(i)}), y^{(i)})$$

$$= -\frac{1}{m}[\sum_{i=1}^{m} y^{(i)} \log h_\theta(x^{(i)}) + (1 - y^{(i)}) \log (1 - h_\theta(x^{(i)}))]$$

To fit parameters θ :

$$\min_\theta J(\theta)$$

Gradient Descent:

$$J(\theta) = -\frac{1}{m}[\sum_{i=1}^{m} y^{(i)} \log h_\theta(x^{(i)}) + (1 - y^{(i)}) \log (1 - h_\theta(x^{(i)}))]$$

Want $\min_\theta J(\theta)$

Repeat $\{$

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$$

(simultaneously update all θ_j)

$\}$

$$\frac{\partial}{\partial \rho} g(z) = \frac{\partial}{\partial \rho}\left(\frac{1}{1+e^{-z}}\right) = \frac{-1}{(1+e^{-z})^2} \frac{\partial z}{\partial \rho} e^{-z}$$

$$= \left(\frac{1}{1+e^{-z}}\right)\left(\frac{e^{-z}}{1+e^{-z}}\right) = \left(\frac{1+e^{-z}-1}{1+e^{-z}}\right)\left(\frac{1}{1+e^{-z}}\right)$$

$$= \left(1 - \frac{1}{1+e^{-z}}\right)\left(\frac{1}{1+e^{-z}}\right)$$

$$\boxed{\frac{\partial}{\partial \rho} g(z) = (1 - g(z)) g(z) \frac{\partial z}{\partial \rho}}$$

$$\frac{\partial}{\partial \theta_j} J(\theta) = -\frac{1}{m} \sum_{i=1}^{m} \left(y^{(i)} \frac{\partial}{\partial \theta_j} \log\left(h_\theta x^{(i)}\right) + \left(1 - y^{(i)}\right) \frac{\partial}{\partial \theta_j} \log\left(1 - h_\theta x^{(i)}\right) \right)$$

Also $\frac{\partial}{\partial x} \log x = \frac{1}{x}$ and $\frac{\partial}{\partial z} g(z) = g(z)(1 - g(z))$

$$\boxed{\frac{\partial}{\partial \theta_j} J(\theta) = \frac{1}{m} \sum_{i=1}^{m} \left(h_\theta x^{(i)} - y^{(i)} \right) x_j^{(i)}}$$

Gradient Descent

Repeat $\{$

$$\theta_j := \theta_j - \frac{\alpha}{m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

(simultaneously update all θ_j)

$\}$

Algorithm looks identical to linear regression, but the hypothesis function is different for logistic regression.

Underfitting and Overfitting in ML

- Machine learning models should learn useful patterns from training data. When a model learns too little or too much, we get underfitting or overfitting.
- Underfitting means that the model is too simple and does not cover all real patterns in the data.
- Overfitting means that the model learns not just the underlying pattern, but also noise or random quirks in the training data. model memorizes training data
- A good model finds the right *spot, it is* complex enough to capture real patterns, but not so complex that it “memorizes” noise.

What is Underfitting?

- Underfitting happens when the model fails to learn important patterns. It performs poorly on both training and testing data.
- Underfitting happens due to:
  - Model is too simple
  - Very high regularization
  - Features are weak or missing
  - Not enough training
  - High bias

What is Overfitting?

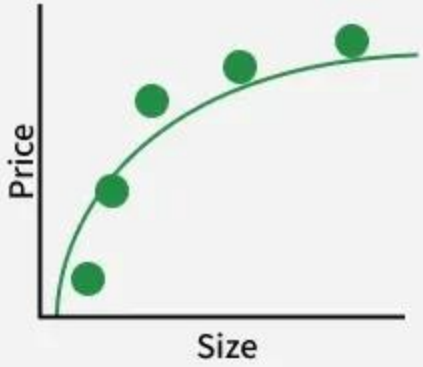
- Overfitting happens when the model learns too much from the training data, including noise and outliers. It performs very well on training data but poorly on test data.
- Overfitting happens due to:
  - Model too complex
  - Too many features
  - Very little data
  - No regularization
  - High variance



$\theta_0 + \theta_1 x$

High Bias

(Underfitting)



$\theta_0 + \theta_1 x + \theta_2 x^2$

Low Bias, Low Variance

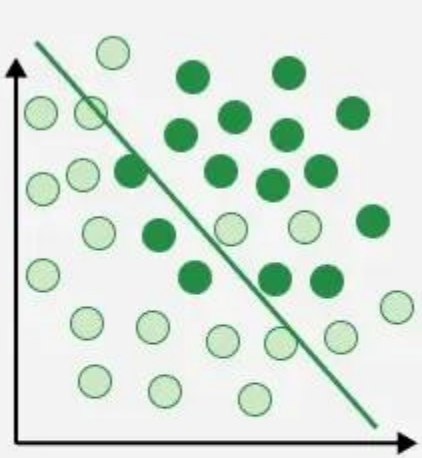
(Goodfitting)



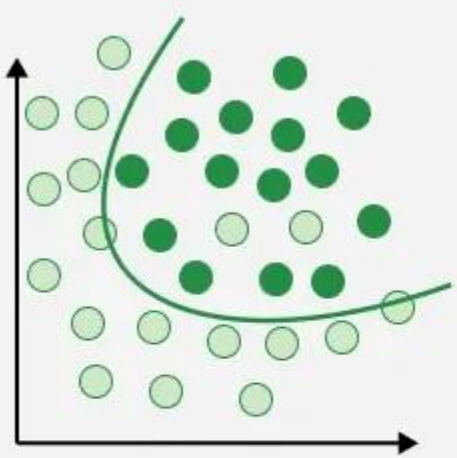
$\theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4$

High Variance

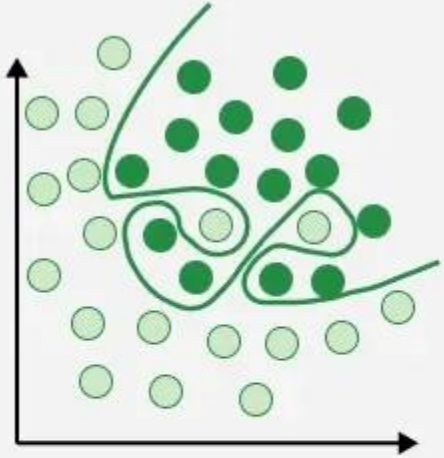
(Overfitting)



Under-fitting



Appropirate-fitting



Over-fitting

Techniques to Reduce Underfitting

- Use a more complex model
- Add new features and perform feature engineering
- Reduce regularization
- Train for more epochs
- Scale features properly

Techniques to Reduce Overfitting

- Collect more training data
- Reduce model complexity
- Use regularization
- Apply dropout (for neural networks)
- Use early stopping
- Clean noisy data


Thank You